

Custom Web Scraper

The Brief: what it is, how it is built, where it is weak

Portfolio explainer, built from the source

In one paragraph

The project

A desktop application that opens a real Chrome browser, loads a web address you type, and then lets you interrogate that live page with any of seven Selenium locator strategies, showing every match in a table and outlining them in the browser itself. It exists because small scraping investigations do not deserve a throwaway Python script each: load the page once, then fire locator after locator at it from a graphical interface. Roughly 550 lines of Python in one file, with exactly one third-party dependency.

1 Why it is built the way it is

Two decisions define the project, and both are defensible.

It drives a real browser instead of downloading HTML. The simple approach, fetching a page over HTTP and parsing the text, sees only the initial response. Most modern sites build their content afterwards with JavaScript running inside the browser, so that approach returns an empty shell. Selenium drives actual Chrome, so the page is fully rendered before anything is read. The price is a Chrome install and a window on your screen, and the project accepts it knowingly.

It is a GUI, not a library. The target user is someone poking at an unfamiliar page for five minutes. The interface encodes that workflow directly: panel “1. Load a page”, panel “2. Find elements”. The order of operations is the layout.

2 Architecture

Three layers live in one file, separated by convention rather than by folders.

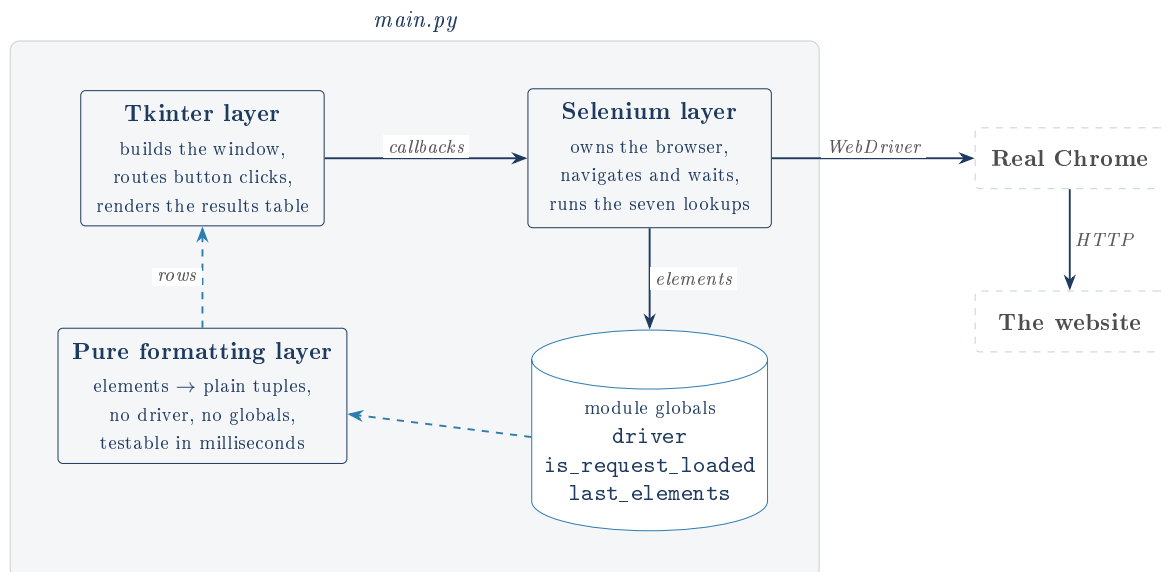


Figure 1: Three layers, three shared globals, two things outside the process.

The **pure formatting layer** is the part worth pointing at. It converts Selenium’s live element handles into flat display tuples, and it does so without touching the driver or any global: it only calls `.tag_name`, `.text` and `.get_attribute()` on whatever object it is handed. Any object with those three members will do, so it can be tested with a five-line fake class, no browser and no network. That is a real design decision, and it is the strongest thing in the codebase.

The rest of the file cannot make that claim, and the reason is the box in the middle of the diagram: `driver`, `is_request_loaded` and `last_elements` are module-level globals that six functions reach into directly.

3 How one lookup flows

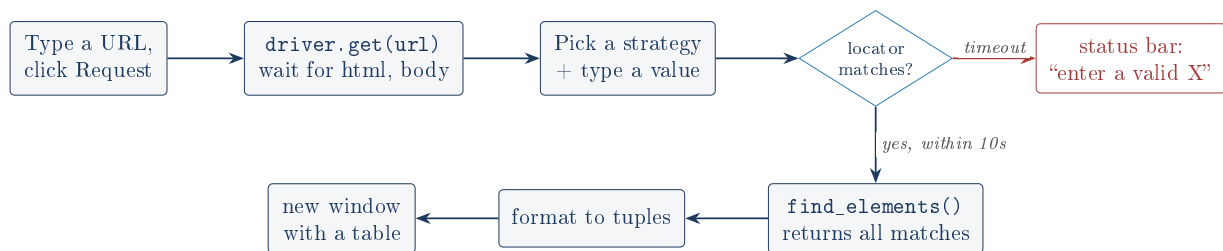


Figure 2: The path of a single lookup. The “timeout” branch also fires when a perfectly valid locator simply matches nothing, which is the source of the mislabelled-error defect in Section 5.

The seven strategies are class, ID, name, tag name, link text, partial link text and XPath, held in a `LOCATOR_DISPATCH` dictionary keyed by the dropdown’s label. The dropdown’s initial value, “Select”, is deliberately absent from that dictionary, so a missing key doubles as the “you have not picked one yet” check with no second list to keep in sync.

Each lookup waits before it finds. A ten-second `WebDriverWait` on `presence_of_element_located` blocks until at least one match exists, and then `find_elements` collects all of them. This is the correct pattern for a page that may still be rendering, and the project uses it consistently rather than sleeping and hoping.

4 Features of substance

- **Highlight in Browser.** After a lookup, this injects a few lines of JavaScript into the live page via `execute_script`, outlining each matched element in pink and scrolling it into view. It closes the loop between “my locator matched 12 things” and “are they the 12 I meant?”, which is the entire question a locator-testing tool exists to answer. The best feature here.
- **Status bar.** A single line, recoloured by severity from a three-key dictionary, reporting every step. A `root.update_idletasks()` call forces the “Loading page...” repaint before the browser blocks the event loop, so the window does not simply appear to freeze.
- **Session history.** Both text fields are comboboxes backed by one shared 10-entry most-recently-used list. In memory only; it dies with the process, which the README states plainly.
- **Contextual instructions.** A label under the dropdown rewrites itself per strategy through a `StringVar` trace callback.
- **Portable driver resolution.** `CHROMEDRIVER_PATH` points at a specific binary; unset, Selenium Manager fetches the right one automatically. This replaced a driver path hardcoded to a Windows folder on the original author’s machine, which meant the project ran on exactly one computer on earth. It is the best “portability problem I solved” story here.

5 Where it is weak

The README claims a test suite the repository does not contain

The README states that a 7-case unit suite and a full window test pass, and refers the reader to a “Test plan” section for the command and output. That section does not exist, and no test file has ever been committed: `git ls-files` lists eight files, none a test, and a search of every commit for every file ever added confirms it was never there.

The claims themselves appear to be true. Equivalent tests written from scratch for this document reproduced every stated behaviour. So the tests were evidently written and run but never committed, and the dangling cross-reference is what exposes it. This is the project’s most urgent problem, and it is not in the code: an interviewer who reads “all 7 cases pass; see Test plan below” will go looking and find nothing. Commit the tests or drop the claim.

Two further README claims the code contradicts

The results table is not sortable, though the opening paragraph says it is. `ttk.Treeview` sorts only when a command is bound to each column heading, and none is bound.

`temp.py` and `temp1.py` are not “still visible in the source history”. The BeautifulSoup draft they refer to may have existed locally, but this repository has four commits and has never contained a file by either name.

Real defects in the code

- **“No matches” is reported as “please enter a valid class name”**. Because the wait must succeed before `find_elements` runs, a locator that matches nothing always times out first. The zero-match handling that follows, the count of 0, the disabled button, the “No elements matched” label, is therefore unreachable, and a valid locator gets blamed on a typo the user did not make. Confirmed by forcing the branch with a stubbed wait: the code is correct, just unreachable.
- **`is_request_loaded` is set before the load is confirmed**, so a page-load timeout leaves the app believing a page is ready when it is not.
- **Only `InvalidArgumentException` is caught around `driver.get`**, so a dead host or crashed browser escapes as a raw traceback while the GUI sits on “Loading page...”. For a tool whose selling point is never failing silently, that is the gap.
- **Stale highlight state**. `last_elements` and the highlight button are never reset on navigation or after a failed lookup, so the button stays enabled pointing at a previous page’s elements.
- **Every lookup opens a new window** and none is ever closed, so the core workflow of trying locator after locator is also what buries the screen.

The globals, and the seven near-identical functions

The seven `get_element_by_*` functions differ only in one `By` constant and one error string; the guard, the wait, the exception handler and the return shape are copied seven times. `LOCATOR_DISPATCH` removed the `if/elif` chain in the caller but left the duplication in the callees.

All seven read `driver` from module scope rather than receiving it, so none can be tested without patching the module, and the app structurally cannot hold two sessions. `driver` is not even given a module-level default; it exists only once the `__main__` block assigns it, so

importing the module and calling anything raises `NameError` until a fake is patched in. The README says all of this about itself, unprompted, which counts for something. The fair framing is not “this is wrong” but “this traded testability for brevity in a 550-line single-window tool, knowingly”. The fix is a small class holding `driver` and `is_request_loaded` as instance attributes.

6 What it gets right

- The pure formatting layer, and specifically the choice to wrap *each field access* in a small exception-swallowing helper rather than the whole row, so an element going stale mid-read still yields a partial row. That granularity shows the stale-element problem was understood, not just caught.
- Truncation that is actually correct: 200 characters in gives exactly 80 out, ellipsis included in the budget rather than pushed past it. Verified.
- A dependency range chosen for a reason: `selenium>=4.15,<5`, floored above the 4.6 release that introduced Selenium Manager, ceilinged below the next major version’s licence to break the API.
- Toolkit fluency in small places: `update_idletasks` before a blocking call, `rowconfigure` weights so the table absorbs resize, the `clam` theme wrapped in a `try/except` because it is not on every platform.
- One genuinely subtle guard, `if "instructions" not in globals(): return`, which looks like dead paranoia and is not: constructing the dropdown writes its variable and fires the trace callback immediately, tens of lines before the label it updates is created. Without it, startup raises `NameError` every time.

7 Verified for this document

- `py_compile` clean on Python 3.12.3 with selenium 4.45.0.
- The formatting functions driven with fake duck-typed elements: truncation, whitespace collapsing, attribute filtering, 1-based numbering, and a fully stale element yielding (3, '?', ','). All as described.
- The entire window built and driven on a real display against a fake driver: load, lookup, results table, and highlight all produced the expected status text and the expected 2 `execute_script` calls.
- Repository history searched: no test file, no `temp.py`, ever.

Not verified: a live Chrome session against a real site, and the highlight JavaScript against a real DOM. Both need Chrome plus network access for Selenium Manager. The Selenium API used is standard 4.x, but that claim rests on inspection, not execution.

Glossary

DOM

The browser’s live in-memory tree of a page, built from its HTML. What Selenium actually queries.

Element

One node of that tree: a link, a paragraph, an image.

Locator

A rule for finding elements, in two halves: a strategy (by class? by ID?) and a value (which class?). The split the whole interface is built on.

Selenium / WebDriver

A library, and the standard protocol it speaks, for remote-controlling a real browser from code.

Selenium Manager

Bundled since selenium 4.6; detects your Chrome and downloads a matching driver, removing the version-matching chore by hand.

Tkinter / ttk

Python's built-in GUI toolkit, and its themed widget set.

Event loop

The endless wait-for-input, call-the-callback cycle a GUI runs in. `root.mainloop()` enters it and does not return until the window closes.

Callback

A function registered to run when something happens, such as a button being clicked.

Duck typing

Python's rule that only an object's methods matter, not its type. What makes the formatting layer testable with fakes.

Stale element reference

The error raised when you read an element handle after the page has re-rendered underneath it. Routine on modern sites.

XPath

A query language for walking a document tree, and the escape hatch that expresses what the other six strategies cannot.

Headless

Running the browser with no visible window. Enabled here by setting `SCRAPER_HEADLESS=1`.